Technical Environment: Returns and Refunds Module — OrderFlow Platform

Author: Raja SP, AWS Labs
Source: https://github.com/awslabs/aidlc-workflows
Adaptation: Ricardo de Luna Galdino, EngSoft Learn
Download: example-minimal-tech-env-brownfield.pdf

> **Brownfield project.** The existing stack is the baseline. New code must fit into the established patterns. Where a choice is not listed below, follow the existing codebase — do not introduce new patterns without justification.

Existing Stack (must be preserved)

Layer	Current Technology	Version	Notes
Language	TypeScript	5.x	Strict mode. Do not introduce JavaScript files.
Runtime	Node.js	20.x LTS	
API framework	Express	4.x	All existing services use Express. Do not introduce Fastify or Koa.
Database	PostgreSQL	15	Via pg and node-postgres. No ORM — raw SQL with typed query helpers.
Infrastructure	AWS ECS Fargate	—	Services deploy as Docker containers. CDK for all infra.
Message bus	Amazon SQS	—	Used by notification-service for async email dispatch.
Auth	AWS Cognito	—	JWT tokens validated at API Gateway. Do not build a new auth layer.
Package manager	npm	10.x	Do not introduce yarn or pnpm.
Test framework	Jest	29.x	With ts-jest. All tests in `__tests__/` alongside source.
Linter / formatter	ESLint + Prettier	—	Config files are in the repo root. Do not modify them.

What to Add (new for this module)

- A new `returns-service` following the same structure as `order-service`
- New PostgreSQL tables: `return_requests`, `return_items`, `return_status_history`
- New React components for the customer return form and operations dashboard

- These additions must not modify existing tables or service contracts

---

What to Keep Unchanged

- `order-service`, `payment-service`, `notification-service` — do not modify these services
- Existing PostgreSQL tables — additive migrations only (new tables, new columns on new tables)
- The `notification-service` API contract — call it as documented, do not extend it
- Existing CDK stacks — add a new stack for `returns-service`, do not edit existing stacks
- Frontend design system components — use existing components, do not create replacements

---

What to Remove / Not Introduce

Prohibited	Reason	Use Instead
ORMs (TypeORM, Prisma, Sequelize)	Existing codebase uses raw SQL with typed helpers. Introducing an ORM creates inconsistency.	node-postgres with typed query functions, matching existing pattern
Axios	Project uses native fetch (Node 20 built-in).	fetch
Any new CSS framework	Existing frontend uses Tailwind CSS.	Tailwind CSS, existing design system components
New state management library	Existing frontend uses React Context + useReducer.	React Context + useReducer
New test runner (Vitest, Mocha)	Project uses Jest throughout.	Jest
Separate auth service or middleware	Auth is handled at API Gateway via Cognito JWT.	Validate the JWT passed in the Authorization header, same as other services

---

Security Basics

- Authentication: Cognito JWT validated at API Gateway. Services receive `x-user-id` and `x-user-role` headers — trust these, do not re-validate the JWT in the service
- Authorization: Operations dashboard endpoints require `role === 'operations'` — check this header
- Input validation: Validate all request bodies with Zod schemas before processing
- PII: Return requests contain customer names and addresses — do not log these fields
- Secrets: Database credentials and service URLs via AWS Secrets Manager, same as existing services

---

Example Code Patterns

Follow these patterns from the existing codebase. Do not invent alternatives.

A service endpoint (Express route handler):

```
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { createReturnRequest } from '../domain/returns';
import { AppError } from '../errors';

const router = Router();

const CreateReturnSchema = z.object({
  orderId: z.string().uuid(),
  items: z.array(z.object({ orderItemId: z.string().uuid(), reason: z.string().min(1) })).mi
n(1),
});

router.post('/returns', async (req: Request, res: Response) => {
  const parsed = CreateReturnSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(400).json({ error: 'VALIDATION_ERROR', details: parsed.error.flatten()
});
  }
  try {
    const result = await createReturnRequest(parsed.data, req.headers['x-user-id'] as strin
g);
    return res.status(201).json(result);
  } catch (err) {
    if (err instanceof AppError) {
      return res.status(err.statusCode).json({ error: err.code, message: err.message });
    }
    throw err;
  }
});

export default router;
```

A database query function:

```
import { pool } from '../db/pool';

export interface ReturnRequest {
  id: string;
  orderId: string;
  customerId: string;
  status: 'submitted' | 'approved' | 'rejected' | 'refunded';
  createdAt: Date;
}

export async function getReturnRequestById(id: string): Promise<ReturnRequest | null> {
  const { rows } = await pool.query<ReturnRequest>(
    'SELECT id, order_id AS "orderId", customer_id AS "customerId", status, created_at AS "c
reatedAt" FROM return_requests WHERE id = $1',
    [id]
  );
  return rows[0] ?? null;
}
```

A Jest test:

```
import { getReturnRequestById } from '../db/return-requests';
import { pool } from '../db/pool';

jest.mock('../db/pool');
const mockQuery = pool.query as jest.Mock;

describe('getReturnRequestById', () => {
  it('returns the request when found', async () => {
    mockQuery.mockResolvedValueOnce({ rows: [{ id: 'abc', orderId: '123', status: 'submitte
d' }] });
    const result = await getReturnRequestById('abc');
    expect(result?.id).toBe('abc');
  });

  it('returns null when not found', async () => {
    mockQuery.mockResolvedValueOnce({ rows: [] });
    const result = await getReturnRequestById('missing');
    expect(result).toBeNull();
  });
});
```